

aidev

Complementary Agents for Autonomous Software Maintenance at Portfolio Scale

An experience report from the aidev runner, 2026-09-02 to 2026-09-19

버전 draft v1

작성일 2026-09-19

문서 Complementary Agents for Autonomous Software Maintenance at Portfolio Scale

Complementary Agents for Autonomous Software Maintenance at Portfolio Scale

An experience report from the aidev runner (2026-09-02 → 2026-09-19)

hkjang · with Claude (Fable 5.1) as co-author of the system and of this report · draft v1, 2026-09-19 ·
Korean version: [한국어](#)

Abstract

We report on eighteen days of operating **aidev**, a runner that autonomously maintains a portfolio of 46 small-to-medium repositories owned by one person. In each *round* the runner checks out a repository at a pinned commit, has an LLM agent choose and implement one improvement, verifies the change itself, opens a pull request, gates the merge on CI and an independent review, merges, and releases according to the repository's own conventions. Over the period the runner performed 1,031 rounds, opened 443 pull requests, merged 244 of them without a human touching the keyboard, and published 331 releases, at a model cost of about USD 3,750.

The central design question was not "can a model write a patch" but "how do we keep a fleet of unattended patches honest." Our answer evolved from a single all-purpose agent into a set of **complementary agents** that cannot approve their own work: a read-only *scout* that writes the task brief, a *builder* that implements it, a *critic* that reads only the diff, a *repairer* that fixes only what the critic named, an *arbiter* that settles disagreements, a *releaser*, an hourly *shepherd* pair that unsticks pull requests, and *historians* that distill rejections and regressions into rules the other agents read before they start. The agents collaborate through a per-round journal in which each role records what it did and, importantly, what it is unsure about.

We describe the architecture, the safety envelope that made unattended operation tolerable (pinned bases, runner-side verification, secret and artifact gates, protected paths, autonomy levels with automatic demotion, SHA-pinned approvals, budgets, kill switches), the *campaign* mechanism for rolling one feature across thirty repositories with a written standard and a reference implementation, and the measurements we have. We then list the failures that actually happened and the operational know-how they produced. Most of the hard-won lessons are not about prompting; they are about plumbing: stdin consumed inside loops, scheduler processes killed mid-round, a tab-separated field that closed four campaigns at once, and notifications that looked like an intrusion.

1. Introduction

A single developer who owns forty repositories cannot give each of them attention every week. Dependencies rot, guides drift away from the code, the same feature (single sign-on, mail notifications, a tracking snippet, an MCP endpoint) has to be added to thirty services in the same shape, and every one of those changes needs a test, a review, a merge and a release. The interesting property of this workload is not that any single change is hard. It is that there are too many of them, they are mostly routine, and the cost of a wrong one is real: a broken release, a leaked secret, an authentication bypass.

aidev is an attempt to hand this workload to LLM agents while keeping the operator in control of *what may be merged unattended*. It began on 2026-09-02 as a shell script that ran one agent per repository and grew, through daily operation and a long list of incidents, into a system with eleven named agent roles, a simulation harness, a dashboard, a Telegram interface and a set of written standards.

This report makes four contributions.

1. **An architecture for complementary agents** in which the roles that make changes and the roles that judge changes are different sessions with different tools, and no role can approve its own output (§3).
2. **A safety envelope for unattended merging** built from cheap mechanical gates rather than from model judgment alone (§4).
3. **Campaigns**: a way to roll one feature across many repositories using a written standard and a reference implementation, with campaign-level lessons distilled from the first repositories' failures and injected into the later ones (§5).
4. **Measurements and incidents** from 1,031 rounds, including what the numbers do and do not support (§6, §7), and the operational know-how we would give anyone building the same thing (§8).

Everything described here is in the repository `hkjang/aidev` (runner `bin/run.sh`, prompts in `prompt.md`, `review-prompt.md`, `agents/*.md`, documentation in `AGENTS.md` and `README.md`), and the raw data behind the numbers is published as `docs/data/runs.jsonl` and `docs/data/usage.jsonl`.

2. The system

2.1 A round

A round is the unit of work. The runner (a Bash script driven by a Windows Task Scheduler trigger every ten minutes, serialized by a file lock) picks a repository, then:

1. **Pins the base**. It fetches the default branch, records its SHA, and creates a fresh git worktree from that SHA. Every later judgment (diff, guard, verification) is made against this pinned base, and a merge is refused if the base moved without a re-verification.

2. **Runs the agents** (§3) inside the worktree with an isolated home directory, no GitHub credentials, and a process-local push block. The agents can build and run tests; they cannot push, cannot read the operator's secrets, and cannot see the runner's policy files.
3. **Verifies the change itself.** The runner runs the project's test, lint and build commands (from policy or auto-detected: `go test ./...`, `npm test`, `pytest`, `gradlew test`, and so on), scans the diff for secrets, and refuses committed build artifacts. This is independent of anything the agent claims.
4. **Opens a pull request** with the agent's ledger entry and, since §3.5, the round journal.
5. **Gates the merge.** Protected paths (workflows, migrations, auth, payments, secrets, LICENSE) hold the PR for a human. Otherwise CI must pass twice in a row, and an independent critic must approve.
6. **Merges and releases.** A release agent reads the repository's previous releases and repeats their convention (tag, version file, changelog, workflow, attached assets). The runner verifies the release: the tag exists, the GitHub Release exists, the required assets are present and match a manifest derived from the previous release.
7. **Records.** Every round produces a structured record (`runs.jsonl` : outcome, stages with state and reason, PR, SHAs, campaign) and an evidence directory (`state/runs/<id>/`), and the dashboard is regenerated.

2.2 Outcomes

Rounds end in one of eight outcomes: `release-ready` (merged and a verified release published), `merged` (merged, nothing to release), `releasing` (merged, release incomplete), `review-pending` (PR open and waiting), `verify-failed` (the runner's own verification or CI failed), `no-change`, `error`, and later `infra-error` and `usage-limit` for failures that are not the repository's fault. The distinction between the last two and `error` turned out to matter for campaigns (§5.3).

2.3 Around the round

Several processes run outside rounds: a health check every 30 minutes that restarts a dead scheduler and unwedges locks; a regression watcher that checks main CI and reverts 2 - 48 hours after each merge; a *fixer* track that re-queues rounds that ended in error; the hourly *shepherd* (§3.4); the historians (§3.6); and a Telegram copilot through which the operator can ask "why is weekly stuck" or say "approve moira's PR" from a phone.

3. Complementary agents

3.1 Why one agent was not enough

For the first two weeks a single "improve" agent did everything: read the repository, list ideas, pick one, implement it, run the tests, commit, and write the ledger. It worked, in the sense that it produced merged releases from day one. Its failure modes were consistent, though:

- **It chose what was easy to finish, not what was worth doing.** An agent that must also implement the idea in the same session is biased toward ideas it is confident it can complete.

- **It graded its own homework.** The ledger said "verified", and often the verification had been run; but the runner's independent verification still failed in 11% of rounds that produced a commit, and the independent critic rejected 18% of the diffs that reached it.
- **It re-learned the repository every time.** Roughly the first ten minutes of a 45-minute session were spent reading README, layout, test configuration — identical work, forty times a day.
- **When it was wrong, nobody argued back.** A rejected review left a PR open with a comment. Nothing read the comment.

The restructuring into complementary roles (2026-09-18) was driven by these observations and by one principle: *the role that makes a change and the role that judges it must be different sessions, and the judge must not be able to edit.*

3.2 The roster

Role	Session tools	Produces	Who checks it
Scout	read-only; may run tests; cannot commit (any leftover change is discarded by the runner)	brief.md — task, acceptance criteria, files to touch, the exact verification command, risks, a fallback candidate; a refreshed idea backlog; the project profile	the builder, who must confirm the brief against the code and may record 채택 / 차선 / 기각 (adopted / fallback / rejected)
Builder	edit, run, commit	commits, ledger entry, ideas	runner verification, then the critic
Critic	read and run; cannot edit	review.json — verdict, reasons with file:line, risk	a schema gate; only approve reaches the merge path
Repairer	edit, commit (only what the critic named)	a repair commit, fix-summary.md	re-verification, then the critic again; failed repairs are reset
Arbiter	read and run; cannot edit	arbiter.json — per-reason "confirmed / refuted"	runs only when the repairer declined to change anything and left a rebuttal
Releaser	edit, tag, build assets	release.json , tag, assets	release gate, asset manifest, workflow result
Shepherd fixer / reviewer	as repairer / as critic, on the PR branch, hourly	fix commits; review.json with a recommend field	runner verification; the approval sweep, which merges only the approved SHA after CI
Historians	none (offline distillation)	campaign-lessons/<id>.md , operator-preferences.md	input-hash cache; humans read them on the dashboard
Planner	read-only + write to drafts/	a campaign draft: standard, targets, budget	a human activates it
Copilot	one script's verbs only	operational actions	the operator, in the chat

The scout and the critic are the two roles that cannot change code. That asymmetry is the point. The scout, unable to implement, picks the task on merit; the critic, unable to edit, cannot "fix and approve".

3.3 The in-round loop



The critic runs *before* the PR is opened, but only when the change would otherwise be auto-merged (autonomy `low-risk` or higher, no protected paths). Changes that touch protected paths skip the in-round critic because a human or the shepherd reviewer will look at them anyway; running the critic there would double the cost for no decision. If the loop ends without approval, the PR is opened and held with the critic's reasons, and the shepherd picks it up an hour later in a fresh session — deliberately a different session, so that one model's stubbornness does not persist.

3.4 After the PR: the shepherd

By 2026-09-17 there were 191 pull requests that the runner had opened and nobody had looked at: 55 held by protected paths, 52 rejected by the critic, 25 with failing CI, 16 with no CI at all, 15 with merge conflicts. The operator asked for "a separate process that, every hour, figures out the problem and gets them merged and released."

The shepherd diagnoses each stuck PR from the round's recorded stages and the live GitHub state, then acts:

- conflicts → rebase on a worktree, re-verify, force-push with lease;
- CI failures and review rejections → a fixer session on the PR branch with the failure log or the rejection reasons, runner verification, push;
- protected paths and autonomy holds → straight to a stricter reviewer session (`shepherd-review-prompt.md`), which stands in for the human and therefore *rejects when unsure*.

The reviewer returns a `recommend` field — `merge`, `fix` or `human` — and the runner follows it literally; the operator's instruction was "let the approving agent act on its own recommendation," and we removed the runner-side risk threshold that had overridden it. A `merge` recommendation is recorded in `approvals.jsonl` with `by: shepherd` and the exact SHA, and the ordinary approval sweep merges *that SHA only* after CI. The shepherd never calls `gh pr merge` itself.

Things the shepherd will not touch, by construction: PRs that modify workflows, secrets, payment code or LICENSE; projects whose autonomy is `approve` or lower; commits with no CI at all; anything it has already failed twice on. Those go to the inbox as "needs a human", with the diagnosis attached. In its first two days it took 142 actions: 44 rejections, 28 fixes pushed, 25 approvals, 36 handed to the human (§6.4).

3.5 How the roles talk: the round journal

Artifacts (`brief.md` , `review.json` , `fix-summary.md`) are the *contracts* between roles. They were not enough for *collaboration*: the critic did not know what the builder had been unsure about, the releaser did not know what the critic had worried about even when approving, and the shepherd started every PR cold.

Since 2026-09-19 every round has a single `journal.md` . Each role appends a short section — the scout why it chose this task and what it guessed; the builder what it changed, **what it is not sure about and did not verify**, and what it deliberately left out; the critic what it checked, what it could not see, and what worries survive an approval; the repairer which critiques were right and which were wrong; the arbiter who was right and why. The runner interleaves its own stage verdicts in the same file. The next role is told to read the journal first; the critic specifically is told to start from the builder's "not sure" list, and that an empty list means "I am sure of everything — test that."

The journal travels: it is folded into the PR body, it is part of the change summary given to the releaser, and the shepherd inherits the journal of the round that opened the PR. Two further feedback edges close loops across rounds: the builder's verdict on the brief (`adopted` / `fallback` / `rejected`) is shown to the next scout as "how your last six briefs fared", and the arbiter's overturns are shown to the next critic as "cases where you were too strict here".

3.6 Learning across repositories: the historians

Two offline distillers turn evidence into rules that the working agents read before they start.

Campaign lessons. For each campaign, the historian collects the critic's rejection reasons, the shepherd's rejections and the post-merge regressions of every repository in that campaign, hashes them, and — only when the hash changed — asks a model to distill at most eight *general* rules ("before X, confirm Y in the source", not "in repo Z, line 40"). The rules are injected into the campaign note of every subsequent round and into the shepherd reviewer's hold note. The first distillation (2026-09-17) produced 25 rules from 36 pieces of evidence; one of them — "do not apply byte offsets from a lower-cased copy to the original string; test with U+0130 and U+212A" — had been found independently, and expensively, in two repositories before the rule existed.

Operator preferences. The same mechanism applied to the operator's own words: reasons attached to human rejections (the runner now records the operator's last PR comment into the lesson), emergency-stop reasons, and instructions given to the copilot. Ten rules came out of nineteen pieces of evidence, including "prove behavior through the production wiring, not through hand-injected

doubles," and "changes that do not alter observable behavior are grounds for rejection". These are prepended to the builder's and reviewer's prompts, so the operator does not have to say the same thing twice.

4. The safety envelope

Unattended merging is only acceptable if the cheap, mechanical checks are strong enough that the expensive, fallible ones (model judgment, human attention) are not the last line of defence. audev's envelope, in order of application:

- **Pinned base and fresh worktree.** Nothing is judged against a moving target: if the base moves before merge, the branch is rebased and re-verified, or held.
- **Runner-side verification.** Tests, lint and build are run by the runner, not reported by the agent. A build-artifact scan refuses binaries over 1 MB (documentation PDFs over 15 MB), because one round once committed a 20 MB executable.
- **Secret gate** on the diff, the ledger and now the journal. Placeholders and local DSNs are allow-listed after a week of false positives.
- **Protected paths** (`state/default.guard`): workflows, migrations, auth/session/oauth, payments, secrets, LICENSE. Touching them holds the PR for a human — or, since the shepherd, for the stricter reviewer with an explicit never-list (workflows, secrets, payments, LICENSE stay human-only).
- **Independent review** with a schema gate: a malformed or missing verdict is a hold, not an approval.
- **CI gate:** check-runs are fetched for the exact SHA, must be `success` twice in a row (jobs register late), and a repository with no CI is blocked unless policy says otherwise.
- **Autonomy levels** per project — `analyze < pr < approve < low-risk < release` — raised only by a human editing a file, and **lowered automatically** by the runner when a rollback or regression is detected. Two projects were demoted during the period.
- **SHA-pinned approvals.** A human (or the shepherd) approves a specific commit; if the branch changes, the label is removed and the approval must be repeated.
- **Budgets** at three levels: per phase, per round (the round does not start unless the sum fits the day's remaining cap), per day; campaigns and the shepherd have their own, so that the cap on exploratory work does not block finishing work.
- **Kill switches:** `STOP`, `STOP-merge`, `STOP-release`, `STOP-<project>`, `NO-SHEPHERD`, `NO-COPILOT`, checked at every stage boundary.
- **Evidence.** Every round leaves stages, gate verdicts, review JSON, CI snapshots, the journal and a provenance record (policy version, prompt hashes, runner version).

None of these needs a model to be right. That is the design goal: models are used where judgment is needed, and the envelope makes sure a wrong judgment is cheap.

5. Campaigns

5.1 The problem

The operator's repositories share a stack (Go backend, React SPA, Keycloak SSO) and repeatedly need the same capability. Adding silent SSO, a tracking snippet with CSP, SMTP mail notifications, document hand-off between services, complete user guides with screenshots, and OAuth for MCP endpoints each meant the same change in twenty to thirty repositories.

5.2 Mechanism

A campaign is an entry in `campaigns.json`: an id, a goal written as an instruction to the builder, a list of target repositories, a budget, a deadline, and the protected-path fragments the work is *expected* to touch (so the guard still holds the PR but does not alarm on every one). Each goal points at a **standard** document in the aidev repository and at a **reference implementation** — an existing repository where the capability is already done well — with the explicit instruction "do not copy the code; build the same capability in this repository's language and structure."

The runner treats a campaign as a finite job: each target gets one round; a target whose round reached a PR is done; a target that errored is retried at most three times and then reported as stuck; the campaign closes when no target remains or the budget or deadline is exhausted. Campaigns take precedence over exploratory rounds and rotate among themselves so that a later campaign is not starved by an earlier one.

5.3 What happened

Six campaigns ran in the period, 225 rounds in total:

Campaign	Targets	Processed	Done (merged)	PR waiting	Stuck	Spend
guides (user/admin guides with real screenshots)	25	22 (88%)	6	16	2	\$455
silent-sso (OIDC <code>prompt=none</code>)	33	31 (93%)	5	25	0	\$231
tracking (admin-managed snippet with CSP nonce)	33	29 (87%)	11	18	1	\$360
mail (SMTP relay notifications)	33	29 (87%)	2	27	1	\$356
handoff (document transfer between services)	15	12 (80%)	1	11	0	\$128
mcp-oauth (Keycloak tokens on <code>/mcp</code>)	31	28 (90%)	8	20	0	\$373

Two things stand out. First, the rate at which campaigns *reached a PR* was high (80 - 93% of targets in a few days) and the per-target cost was low (USD 7 - 20). Second, the rate at which those PRs *merged* was low, because almost all of them touch protected paths by design — that is what an auth or migration feature is — and were waiting for a human. This is the gap the shepherd was built to close, and the reason the operator accepted AI approval of AI pull requests for these paths under a stricter review.

Three campaign-specific incidents shaped the mechanism. A goal written on multiple lines broke a tab-separated parse so that `until` read as empty and the campaign closed itself as expired the moment it started (fixed by base64-encoding goals). A column mismatch in the same parse made four campaigns report "all targets done" and close at once (fixed by refusing to close on an empty target list). And a worktree that lost its directory but kept a `locked` registration made one repository fail five rounds in a row and get struck from the campaign; the fix distinguished `infra-error` from `error`, so that infrastructure failures do not count as strikes.

6. Measurements

All numbers are from `runs.jsonl` and `usage.jsonl` as of 2026-09-19 (18 days). The first three days predate the structured outcome field, so 249 early rounds are classified from their result string.

6.1 Volume

Repositories touched	46
Rounds (all kinds)	1,031
Improvement rounds	704
Distinct pull requests opened	443
Merges performed by the runner (stage <code>merge done</code>)	244
Human approvals / shepherd approvals feeding the sweep	60 / 25
Distinct releases published	331
Rounds per day (median / max)	58 / 102

6.2 Quality gates

Gate	Passed	Failed / held	Failure rate
Runner verification (rounds with a commit)	447	57	11%
Independent critic	273	59	18%
CI (exact SHA, two consecutive passes)	228	26 failed, 17 no CI, 3 timeout	17%
Protected paths	—	107 held (50 expected by a campaign)	—
Merge conflicts at merge time	—	17 failed, 15 rebased	—

Critic rejections carried 298 distinct reasons: their self-assessed risk was medium in 87 cases, low in 12, high in 4. Of the 273 approved-and-merged changes, **none** has so far been reverted or rolled back (one rollback and two demotions in the period came from release-workflow failures, not from reviewed code — but see §7 on observation windows).

6.3 Size and cost

Median change	7 files, +330 lines
Changes that include test files	93%
Model cost, total	USD 3,746 over 1,343 sessions
by phase	build \$2,825 · critic \$522 · release \$378 · scout \$16 (two days) · repair \$3
per merged change (all costs ÷ merges)	≈ USD 8
Median session length	build 9.2 min (p90 21) · critic 3.2 min · release 3.0 min · scout 3.7 min
Output tokens / cache reads	29 M / 3.9 B

The cost figures are the model's own estimates and the operator is on a flat-rate plan, so they measure *usage*, not invoice. The cache-read count illustrates why per-session context (profile, journal, lessons) is cheap to add: almost all of each prompt is a cache hit.

6.4 The shepherd, days 1 - 2

Action	Count
Diagnosed as: critic rejection / protected path / conflict / CI failure / no CI	59 / 44 / 22 / 9 / 3
Reviewer rejected (fix next hour)	44
Fixer pushed a verified fix	28
Fixer failed	5
Rebased	3
Reviewer approved → approval sweep	25
Handed to the human, with diagnosis	36

Twenty-five approvals in two days against a backlog that had not moved in a week, at a cost of about USD 105, is the single most visible productivity gain of the period. The 36 hand-offs are almost all the intended kind: workflow files, `approve`-level projects, and PRs that failed twice.

6.5 The complementary loop, day 1

The scout/critic/repair loop had run for one day at the time of writing: 11 scout sessions (9 briefs produced, 2 failures), 2 in-round repairs, both successful. This is too little to conclude anything about the loop's effect on the rejection rate; we report it because the mechanism is in place and the scorecard on the dashboard will accumulate it.

7. Threats to validity

- **One operator, one stack.** Forty-six repositories by one person with a shared Go/React/Keycloak stack is a narrow population. The campaigns in particular exploit that homogeneity.
- **Short window, moving system.** Eighteen days, during which the runner changed almost daily. Numbers aggregate several versions of the system.
- **Regression observation is short.** "No reviewed change has been reverted" is measured with a 2 - 48 hour watcher and eighteen days of history. Latent defects in merged code will not show up in these figures.
- **Self-reported cost.** Session costs are model estimates on a flat-rate plan.
- **Human review of AI approvals is thin.** The operator explicitly delegated approval of protected-path PRs to the shepherd reviewer. We cannot yet say whether that reviewer is as good as the human it replaced; the scorecard tracks "approved then regressed" for exactly this reason, and it reads zero so far.
- **The critic and the code come from the same model family.** Independence is procedural (different session, different tools, diff-only view), not epistemic.

8. Incidents and know-how

The list below is what we would hand to anyone building an unattended agent runner. Each item cost at least one wasted day.

8.1 Plumbing, not prompting

1. **Anything that reads stdin inside a `while read` loop eats the loop.** The approval sweep stopped after a few PRs (`gh` consumed stdin, 09-07); the campaign-lessons distiller processed only the first campaign (`claude -p` consumed stdin, 09-17). Redirect `</dev/null` on every subprocess in a loop.
2. **Your scheduler will kill your process.** Windows Task Scheduler ended the WSL wrapper at the next trigger, taking the round with it; rounds over ten minutes never completed for a day. Run the round under `setsid`, let the wrapper wait, and copy the script before executing it so a `git pull` mid-round cannot change the running code.
3. **Serialize writes to your own state repository.** Two rounds doing `git pull --rebase` at once left a `.git/rebase-merge` that blocked every later sync; `--autostash` silently lost a run record. Take a lock, add and commit before pulling, retry without stash.
4. **Do not use repository config to restrict the agent.** `git remote set-url --push DISABLED` in the shared clone also blocked the operator's own pushes. Use process-scoped `GIT_CONFIG_*` environment variables.
5. **Tab-separated is not a serialization format.** A multi-line campaign goal in a TSV closed the campaign as expired; a missing column closed four campaigns as complete. Base64 any free text, and refuse to act on an empty list.
6. **Worktrees get wedged.** A worktree whose directory vanished but whose registration stayed `locked` failed `worktree add` forever. Reset with `unlock` → `remove` → `rm` → `prune` before every add, and record infrastructure failures separately from project failures.
7. **A notification channel must know about the simulation.** The simulation suite runs the real runner with a fake GitHub; its "merged simproj" message reached the real Telegram and was reported as a suspected intrusion. Every side channel needs the isolation flag.
8. `jq 's // treats false as missing`. A boolean scenario flag set to `false` read as `true`. Test with `if .x == false`.

8.2 Gates and judgment

9. **The runner must verify; the agent may not self-certify.** 11% of rounds whose agent reported "tests pass" failed the runner's own run of the same tests.
10. **Independent review earns its cost.** 18% of diffs that passed verification were rejected by a diff-only critic, with concrete reasons (a `setPreview(null)` that unmounted a panel; a lower-cased byte offset applied to the original string; a plaintext-auth path guarded by an always-false condition). None of the 273 approved changes has regressed so far.
11. **Separate "cannot judge" from "judged bad".** A missing, malformed or over-budget review must hold the PR, never approve it.

12. **Do not let a budget hold look like a failure.** 163 of the 198 rounds recorded as `error` were "hold: budget". They polluted failure statistics, campaign strike counts and human attention until they were recorded separately.
13. **Record infrastructure failures as their own outcome.** Otherwise a wedged worktree strikes a repository out of a campaign.
14. **Autonomy must be able to go down without a human.** Automatic demotion on rollback or regression is what made raising autonomy to `release` by default acceptable.
15. **Approve a SHA, not a PR.** A label on a moving branch is not consent.
16. **A reviewer that replaces a human must reject when unsure.** The in-round critic is told "if uncertain, approve and note it"; the shepherd reviewer is told the opposite, because there is no human after it.
17. **Let the judge's recommendation stand.** Overriding a considered `merge` with a runner-side risk threshold produced a second opinion nobody had asked for. The operator removed it.

8.3 Collaboration

18. **Split choosing from doing.** A scout that cannot commit does not pick the easy task.
19. **Ask the builder what it is unsure about, and point the critic there.** An explicit "not verified" list is the cheapest review guidance we found; an empty list is itself a claim to be tested.
20. **Give the disagreement a third voice.** When the repairer says "the critique is wrong", a loop between two fixed positions is worse than one more session that reads both and runs the code.
21. **Persist the repository profile.** Ten minutes of re-reading per session, forty times a day, is a session's worth of budget. A scout-maintained profile with a 14-day refresh removes most of it and gives every role the same picture.
22. **Distill rejections into rules and inject them upstream.** The same defect was found expensively in two repositories before the campaign-lessons rule existed; after it, the rule is in the prompt of every remaining target.
23. **Make the operator's words reusable.** Rejection reasons, stop reasons and chat instructions become preferences that every agent reads. The operator should never have to say "no attribution trailers" twice.
24. **Notify per stage, but dedupe by content.** The same "no CI" message twelve times in a day taught us to suppress identical notifications for six hours; the same rule then hid the copilot's identical answers until it was exempted.

8.4 The human interface

25. **The inbox must say why and what to do.** A list of open PRs is not actionable; a list with the held stage, the reason, the evidence link and the label to apply is.
26. **A phone is enough.** Approve/reject by label, `run:` and `stop:` issues, and now a Telegram copilot that maps sentences to a small fixed set of verbs. The verb list *is* the permission boundary: the copilot session can execute one script and nothing else.

27. **Some things a human must still own.** Workflow files (a secret-exfiltration path), payment code, licenses, and the decision to raise autonomy. The shepherd's never-list encodes this; the dashboard's "needs a human" column is where those land.

9. Related work

This report is an experience report, not a benchmark. The individual mechanisms have precedents: agentic patch generation and repository-level task solving (SWE-agent, AutoCodeRover, and the SWE-bench line of evaluation); self-critique and refinement loops (Self-Refine, Reflexion); role-structured multi-agent software processes (MetaGPT, ChatDev); and LLM-as-a-judge for code review. What we add is operational: a full merge-and-release pipeline run unattended on a real portfolio for weeks, the safety envelope that made that tolerable, the campaign mechanism for cross-repository rollout with distilled lessons, and the incident record. We deliberately do not compare against those systems, because our workload — many small routine changes across many repositories with releases — is not what their benchmarks measure.

10. Future work

- **Evidence for the loop.** Accumulate the scout/critic/repair/arbiter scorecard for a month and test whether in-round repair actually lowers the review-pending backlog and at what cost.
- **Regression predictor.** Use files touched, protected paths, review risk, test count and project health to route each change to auto-merge, shepherd or human, and to choose the observation window.
- **Cheaper roles.** The scout and the critic are read-only and short; they are candidates for smaller models. The registry supports per-role models with automatic fallback, but we have not measured quality at lower cost.
- **Longer memory.** Profiles and lessons are the first two forms of cross-round memory. Per-project "what the last three critics worried about" and per-campaign "what humans rejected" are obvious next ones.
- **A second operator.** Everything here assumes one owner. Preferences, autonomy and approvals would need identity.

Appendix A. Round record (excerpt)

```
{ "ts": "2026-09-17T11:47:03+09:00", "project": "weekly", "outcome": "release-ready",
  "result": "fix-round: merged https://github.com/hkjang/weekly/pull/17, released v0.303.0",
  "run_id": "2026-09-17-110259-weekly-improve", "base_sha": "04fdb9c...", "head_sha": "58851d1...",
  "stages": { "base": { "state": "pinned" }, "verify": { "state": "passed", "reason": "검증 7개 통과 (auto)" },
    "pr": { "state": "created" }, "review": { "state": "approved", "reason": "리뷰 승인 (risk=low)" },
    "ci": { "state": "passed" }, "merge": { "state": "done" }, "release": { "state": "published" },
    "assets": { "state": "verified" } },
  "files": 6, "additions": 212, "deletions": 31, "tests": 2 }
```

Appendix B. Defaults

Setting	Default
Budget per phase (USD)	scout 2 · build 8 (campaign 12 - 18) · critic 4 · repair 6 · arbiter 3 · release 10 · assets 15
Daily caps	cost \$500 · rounds 100 · releases 60 (campaigns and shepherd have their own budgets)
Timeouts	build 45 min · critic 15 min · scout 10 min · release/assets 60 min
Autonomy	release ; demoted one level on rollback/regression
Protected paths	.github/workflows/ , migrations/ , `auth
Shepherd	hourly · 6 PRs per pass · \$150/day · 2 attempts per PR · never: workflows, payments, secrets, LICENSE
Campaign strike rule	3 errors → stuck; infra-error / usage-limit do not count

Appendix C. Glossary

Round — one unit of runner work on one repository. **Brief** — the scout's task description. **Journal** — the per-round file every role appends to. **Guard / protected path** — a regex list of files that hold a PR for human review. **Autonomy** — a per-project ceiling on what the runner may do unattended. **Campaign** — a finite cross-repository job with a standard and a reference implementation. **Shepherd** — the hourly process that unsticks PRs. **Historian** — offline distillation of rejections/regressions into rules. **Copilot** — the Telegram interface mapping sentences to `ops.sh` verbs.

Data: docs/data/runs.jsonl, docs/data/usage.jsonl, state/lessons.jsonl, state/shepherd.jsonl, state/campaigns.json. *Code:* bin/run.sh, agents/, AGENTS.md. *Dashboard:* <https://hkjang.github.io/aidev/>